先进计算与数字工程研究中心

PATTERN RECOGNITION AND MACHINE LEARNING

Lecture 16 – Deep Learning II: Biological Vision and Convolutional Networks



Chizhi Chris ZHANG

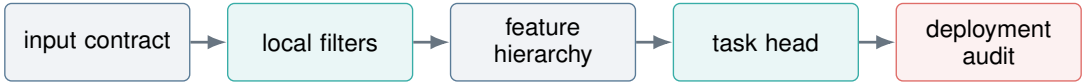
zhangchizhi@ciomp.ac.cn

Advanced Computing and Digital Technology Research Center

University of Chinese Academy of Sciences

Fall 2026

Exploit spatial structure without losing the task



Every spatial reduction is an irreversible claim about what detail the task can spare.

Guiding question

How should locality, sharing, scale, and output resolution shape a visual model?

- images are ordered spatial tensors
- convolution shares local detectors
- downsampling trades resolution for context
- architectures organize stages and shortcuts

From one convolution to a deployable visual hierarchy

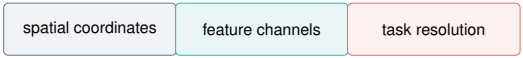
By the end

You can compute convolution outputs and shapes, track receptive fields, explain classic CNN design ideas, construct residual blocks, and validate task-specific vision systems.

Period	Core content	Decision
1	image tensors and convolution	what is shared and where?
2	channels, pooling, and receptive fields	which detail may be discarded?
3	architectures and task heads	which hierarchy fits the task and budget?

Every feature map has coordinates, channels, and a purpose

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER



What to calculate

For each layer, record axis order, spatial size, channel count, effective jump, receptive field, parameter count, activation range, and target hardware latency.

Which statistical regularities make images different from unordered feature vectors?



Question

Which statistical regularities make images different from unordered feature vectors?

Example

recognizing a cat after a small translation in the frame

Main idea

Locality motivates restricted connections; repeated motifs motivate shared detectors; hierarchy combines edges into parts and objects.

Local image patterns motivate convolution

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

对图像，为什么要用CNN？

- 有些模式比整个图像小得多

神经元不需要看整幅图像就能发现模式

与一个小区域连接所需的参数更少



Convolution computes from a local neighborhood

At one spatial location, a convolutional layer combines nearby pixels across input channels:

$$y_{i,j,o} = b_o + \sum_{c=1}^{C_{\text{in}}} \sum_{u,v} W_{u,v,c,o}\, x_{i+u,j+v,c}.$$

- (u, v) ranges over a small kernel, such as 3×3.
- The same weights W are reused at other locations.
- Each output channel o learns one local detection rule.
- Locality and weight sharing are modeling assumptions to test on the task.

Visual locality and translation structure: worked example

Quantity	Value	Meaning
image	224×224	ordered grid
local patch	3×3	edge evidence
same edge	many positions	share one filter

What the result means

Preserve spatial organization until the representation has captured the required local hierarchy.

Visual locality and translation structure: what to check

Main idea

Locality motivates restricted connections; repeated motifs motivate shared detectors; hierarchy combines edges into parts and objects.

What to check

performance under controlled translations and crops

Limit

flattening early discards neighborhood structure and creates a parameter count tied to absolute pixel positions

Visual locality and translation structure: implementation

1. identify spatial axes
2. state expected transformations
3. choose local kernel support
4. share parameters by translation
5. validate across position and scale

Check the result

Check performance under controlled translations and crops on training and held-out examples.

Common mistake

flattening early discards neighborhood structure and creates a parameter count tied to absolute pixel positions

Visual locality and translation structure: common mistakes

Example

recognizing a cat after a small translation in the frame

Explain the calculation and check performance under controlled translations and crops, using a small example.

Common mistake

flattening early discards neighborhood structure and creates a parameter count tied to absolute pixel positions

What to do

Preserve spatial organization until the representation has captured the required local hierarchy.

Visual locality and translation structure: check your understanding

Question

Which statistical regularities make images different from unordered feature vectors?

Answer

Locality motivates restricted connections; repeated motifs motivate shared detectors; hierarchy combines edges into parts and objects.

Decision. Preserve spatial organization until the representation has captured the required local hierarchy.

What exactly does each image axis mean from storage to model input?



A correct shape is not enough: RGB versus BGR, integer versus float, and resize policy can silently change the task.

Question

What exactly does each image axis mean from storage to model input?

Example

moving an image classifier from a notebook to a camera service

Main idea

A correct shape is not enough: RGB versus BGR, integer versus float, and resize policy can silently change the task.

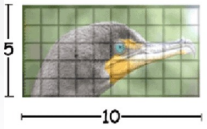
Color images contain several channels

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

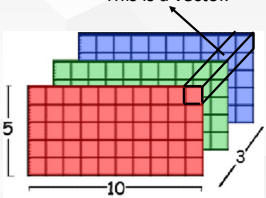
52

应用：图像

An **image** can also be considered as a **vector set**.



This is a vector.



Source of image: https://www.researchgate.net/figure/Color-image-representation-and-RGB-matrix_fig15_282798184

Image tensors, channels, and preprocessing: model and variables

Model and assumptions

An image tensor has batch, spatial, and channel axes whose order, numeric range, color meaning, and normalization form an input contract.

$$X \in \mathbb{R}^{B \times H \times W \times C}, \qquad \widetilde{X}_c = \frac{X_c - \mu_c}{\sigma_c}$$

Symbol	Meaning	Status
B	batch size	independent images
H, W	spatial dimensions	coordinate grid
C	channels	color or modalities

Image tensors, channels, and preprocessing: worked example

Quantity	Value	Meaning
uint8 pixel	$[0, 255]$	stored value
float scale	$[0, 1]$	after conversion
normalize	channelwise	training statistics only

What the result means

Version preprocessing together with model weights and test the served tensor numerically.

The choice rule for Image tensors, channels, and preprocessing

Main idea

A correct shape is not enough: RGB versus BGR, integer versus float, and resize policy can silently change the task.

What to check

range, mean, standard deviation, and label-aligned visual checks

Limit

training on RGB but serving BGR can preserve tensor shape while destroying learned color semantics

Image tensors, channels, and preprocessing: implementation

1. document axis order
2. fix color space
3. define resize and crop
4. fit normalization on training data
5. visualize post-transform samples

Check the result

Check range, mean, standard deviation, and label-aligned visual checks on training and held-out examples.

Common mistake

training on RGB but serving BGR can preserve tensor shape while destroying learned color semantics

Image tensors, channels, and preprocessing: common mistakes

Example

moving an image classifier from a notebook to a camera service

Explain the calculation and check range, mean, standard deviation, and label-aligned visual checks, using a small example.

Common mistake

training on RGB but serving BGR can preserve tensor shape while destroying learned color semantics

What to do

Version preprocessing together with model weights and test the served tensor numerically.

Image tensors, channels, and preprocessing: check your understanding

Question

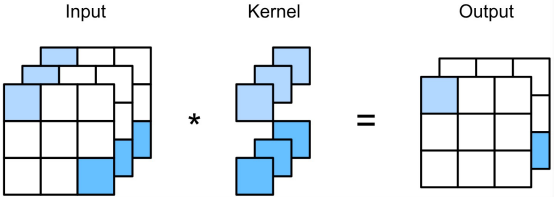
What exactly does each image axis mean from storage to model input?

Answer

A correct shape is not enough: RGB versus BGR, integer versus float, and resize policy can silently change the task.

Decision. Version preprocessing together with model weights and test the served tensor numerically.

How does one local kernel produce a complete feature map?



Question

How does one local kernel produce a complete feature map?

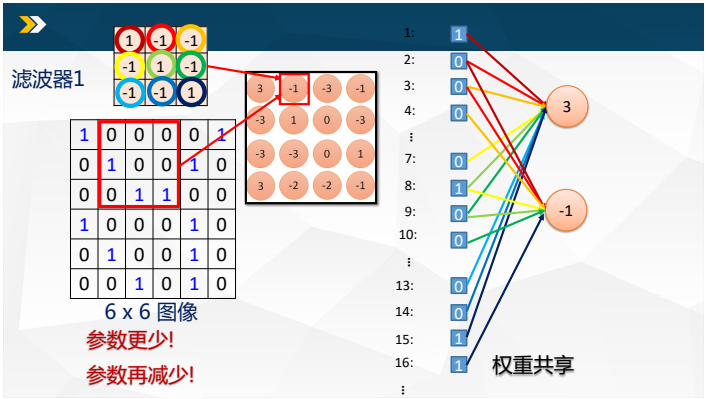
Example

applying a vertical-edge filter to a grayscale image

Main idea

The same weights are reused across positions, turning a local pattern test into a translation-indexed response map.

One filter moves across the image



Discrete convolution and cross-correlation: model and variables

Model and assumptions

A CNN layer slides a shared kernel over spatial neighborhoods and computes a dot product at every valid output position; libraries usually implement cross-correlation.

$$Y_{i,j} = \sum_{a=0}^{K_h-1} \sum_{b=0}^{K_w-1} X_{i+a,j+b} W_{a,b} + \beta$$

Symbol	Meaning	Status
$W_{a,b}$	kernel coefficient	shared parameter
(i, j)	output location	local receptive position
β	filter bias	shared offset

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Discrete convolution and cross-correlation: worked example

Quantity	Value	Meaning
patch	3×3	nine values
edge kernel	signed weights	contrast test
one output	-4	strong opposite orientation

What the result means

State the library convention and verify one coordinate by hand.

Discrete convolution and cross-correlation: checking the result

Main idea

The same weights are reused across positions, turning a local pattern test into a translation-indexed response map.

What to check

manual versus implemented output at selected coordinates

Limit

confusing convolution with cross-correlation flips the kernel and invalidates a hand-derived test

What to record when running Discrete convolution and cross-correlation

```
def conv2d_valid(image, kernel):
    kh, kw = kernel.shape
    out = np.empty((image.shape[0]-kh+1, image.shape[1]-kw+1))
    for i in range(out.shape[0]):
        for j in range(out.shape[1]):
            out[i, j] = np.sum(image[i:i+kh, j:j+kw] * kernel)
    return out
edge_map = conv2d_valid(image, edge_kernel)
print('Valid convolution map:'); print(edge_map)
```

Evidence and failure check

Check manual versus implemented output at selected coordinates with the fitted model and data split; reject the run if confusing convolution with cross-correlation flips the kernel and invalidates a hand-derived test.

The warning sign for Discrete convolution and cross-correlation

Example

applying a vertical-edge filter to a grayscale image

Explain the calculation and check manual versus implemented output at selected coordinates, using a small example.

Common mistake

confusing convolution with cross-correlation flips the kernel and invalidates a hand-derived test

What to do

State the library convention and verify one coordinate by hand.

Explain Discrete convolution and cross-correlation without naming the algorithm

Question

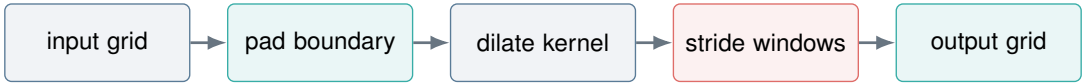
How does one local kernel produce a complete feature map?

Answer

The same weights are reused across positions, turning a local pattern test into a translation-indexed response map.

Decision. State the library convention and verify one coordinate by hand.

Which input coordinates contribute to an output, and how large is the output grid?



Output-shape arithmetic is part of the architecture, not an implementation detail.

Question

Which input coordinates contribute to an output, and how large is the output grid?

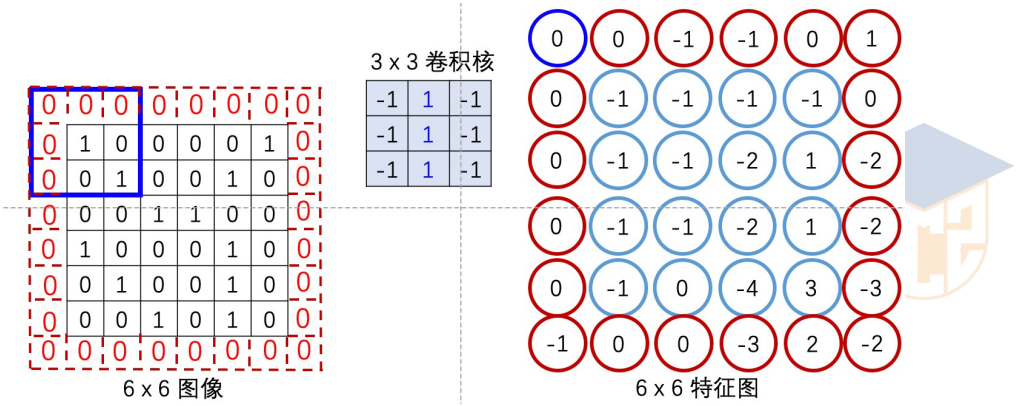
Example

downsampling a feature map while preserving residual compatibility

Main idea

Output-shape arithmetic is part of the architecture, not an implementation detail.

Padding and stride determine output size



Padding changes the border; stride changes how far the window moves.

How Stride, padding, dilation, and output shape changes the representation

Model and assumptions

Stride sets the sampling step, padding extends the boundary, and dilation spaces kernel taps; together they determine output size and alignment.

$$H_{out} = \left\lfloor \frac{H + 2P - D(K-1) - 1}{S} \right\rfloor + 1$$

Symbol	Meaning	Status
P	padding	boundary extension
S	stride	sampling step
D	dilation	kernel tap spacing

Check the direction of change in Stride, padding, dilation, and output shape

Quantity	Value	Meaning
input	$H = 32$	height
kernel	$K = 3, P = 1, S = 2, D = 1$	configuration
output	$H_{out} = 16$	floor rule

What the result means

Calculate shapes symbolically before connecting branches or task heads.

When Stride, padding, dilation, and output shape earns a place in the pipeline

Main idea

Output-shape arithmetic is part of the architecture, not an implementation detail.

What to check

predicted versus actual output shape

Limit

assuming same padding with stride greater than one can create off-by-one alignment errors in skip connections

Stride, padding, dilation, and output shape: implementation

1. write all spatial hyperparameters
2. compute effective kernel size
3. calculate each output axis
4. check boundary alignment
5. assert the runtime tensor

Check the result

Check predicted versus actual output shape on training and held-out examples.

Common mistake

assuming same padding with stride greater than one can create off-by-one alignment errors in skip connections

When the objective of Stride, padding, dilation, and output shape is not enough

Example

downsampling a feature map while preserving residual compatibility

Explain the calculation and check predicted versus actual output shape, using a small example.

Common mistake

assuming same padding with stride greater than one can create off-by-one alignment errors in skip connections

What to do

Calculate shapes symbolically before connecting branches or task heads.

A final decision on Stride, padding, dilation, and output shape

Question

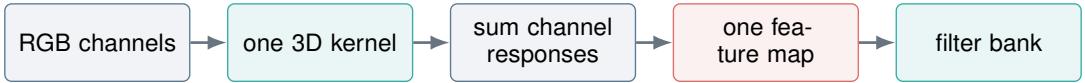
Which input coordinates contribute to an output, and how large is the output grid?

Answer

Output-shape arithmetic is part of the architecture, not an implementation detail.

Decision. Calculate shapes symbolically before connecting branches or task heads.

How does a convolution mix input channels while preserving spatial organization?



Spatial sharing occurs across location, while channel mixing occurs inside each output filter.

Question

How does a convolution mix input channels while preserving spatial organization?

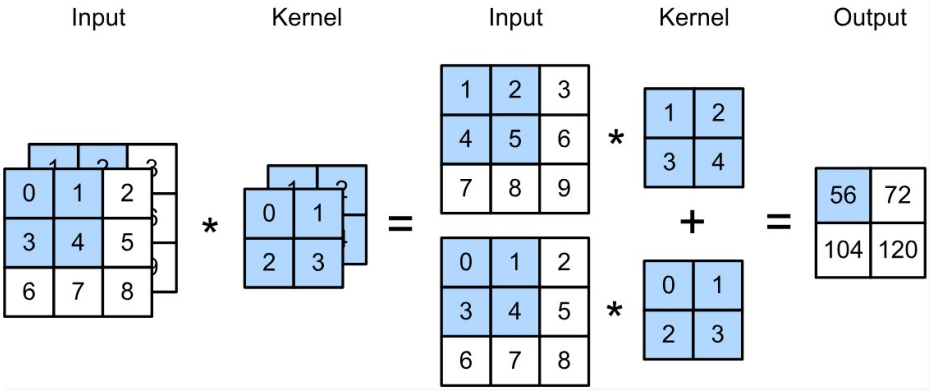
Example

learning color-edge detectors from RGB input

Main idea

Spatial sharing occurs across location, while channel mixing occurs inside each output filter.

A filter combines all input channels



Convolve each input channel and sum its contribution for one output channel.

The variables that make Multi-channel filters and feature banks work

Model and assumptions

Each output channel uses one kernel spanning all input channels; a bank of such kernels produces multiple learned feature maps.

$$Y_{i,j,o} = \sum_{c=1}^{C_{in}} \sum_{a,b} X_{i+a,j+b,c} W_{a,b,c,o} + b_o$$

Symbol	Meaning	Status
C_{in}	input channels	depth of each kernel
o	output channel	one learned detector
$W \in \mathbb{R}^{K_h \times K_w \times C_{in} \times C_{out}}$	filter bank	all kernels

Read the numbers produced by Multi-channel filters and feature banks

Quantity	Value	Meaning
input	$32 \times 32 \times 3$	RGB
filters	64 of size $3 \times 3 \times 3$	weights
output	$32 \times 32 \times 64$	with same padding

What the result means

Let each standard filter span all input channels unless the architecture explicitly uses grouped or depthwise convolution.

Three conditions for trusting Multi-channel filters and feature banks

Main idea

Spatial sharing occurs across location, while channel mixing occurs inside each output filter.

What to check

parameter count, output shape, and channel activation diversity

Limit

treating RGB channels as independent images removes learned cross-channel color combinations

An implementation checklist for Multi-channel filters and feature banks

1. state channel-first or channel-last
2. check kernel depth
3. count parameters
4. inspect output-channel diversity
5. test channel permutation sensitivity

Check the result

Check parameter count, output shape, and channel activation diversity on training and held-out examples.

Common mistake

treating RGB channels as independent images removes learned cross-channel color combinations

Reject Multi-channel filters and feature banks under this condition

Example

learning color-edge detectors from RGB input

Explain the calculation and check parameter count, output shape, and channel activation diversity, using a small example.

Common mistake

treating RGB channels as independent images removes learned cross-channel color combinations

What to do

Let each standard filter span all input channels unless the architecture explicitly uses grouped or depthwise convolution.

Multi-channel filters and feature banks: check your understanding

Question

How does a convolution mix input channels while preserving spatial organization?

Answer

Spatial sharing occurs across location, while channel mixing occurs inside each output filter.

Decision. Let each standard filter span all input channels unless the architecture explicitly uses grouped or depthwise convolution.

How does spatial resolution trade off against context, computation, and localization?



Downsampling increases context per unit and lowers cost, but discards precise location and can alias high-frequency content.

Question

How does spatial resolution trade off against context, computation, and localization?

Example

detecting small lesions in a high-resolution scan

Main idea

Downsampling increases context per unit and lowers cost, but discards precise location and can alias high-frequency content.

Max pooling keeps the largest local response



Pooling, downsampling, and receptive fields: model and variables

Model and assumptions

Pooling or strided layers reduce spatial sampling density, while stacked kernels expand each unit's receptive field in the original image.

$$j_\ell = j_{\ell-1} S_\ell, \qquad r_\ell = r_{\ell-1} + (K_\ell - 1) j_{\ell-1}$$

Symbol	Meaning	Status
j_ℓ	effective jump	input spacing between outputs
r_ℓ	receptive-field size	input context per unit
S_ℓ, K_ℓ	stride and kernel	layer design

Pooling, downsampling, and receptive fields: worked example

Quantity	Value	Meaning
layer 0	$r = 1, j = 1$	pixel
3x3 stride 1	$r = 3, j = 1$	local feature
3x3 stride 2	$r = 5, j = 2$	larger context

What the result means

Match the downsampling schedule to the smallest task-relevant structure.

A decision card for Pooling, downsampling, and receptive fields

Main idea

Downsampling increases context per unit and lowers cost, but discards precise location and can alias high-frequency content.

What to check

accuracy by object size and localization error

Limit

early aggressive pooling can erase small objects before deeper layers have a chance to represent them

Reproduce the result of Pooling, downsampling, and receptive fields

```
def max_pool2(image):
    h, w = image.shape
    return np.array([[image[i:i+2, j:j+2].max() for j in range(0, w-1, 2)] for i in range(0, h-1, 2)])
pooled = max_pool2(image)
print('Pooled shape/value:', pooled.shape); print(pooled)
```

Evidence and failure check

Check accuracy by object size and localization error with the fitted model and data split; reject the run if early aggressive pooling can erase small objects before deeper layers have a chance to represent them.

A stress test for Pooling, downsampling, and receptive fields

Example

detecting small lesions in a high-resolution scan

Explain the calculation and check accuracy by object size and localization error, using a small example.

Common mistake

early aggressive pooling can erase small objects before deeper layers have a chance to represent them

What to do

Match the downsampling schedule to the smallest task-relevant structure.

Pooling, downsampling, and receptive fields: check your understanding

Question

How does spatial resolution trade off against context, computation, and localization?

Answer

Downsampling increases context per unit and lowers cost, but discards precise location and can alias high-frequency content.

Decision. Match the downsampling schedule to the smallest task-relevant structure.

Which ideas from early CNN families remain useful architectural principles?



The durable lesson is staged spatial reduction with growing feature depth, not copying historical input sizes or classifiers.

Question

Which ideas from early CNN families remain useful architectural principles?

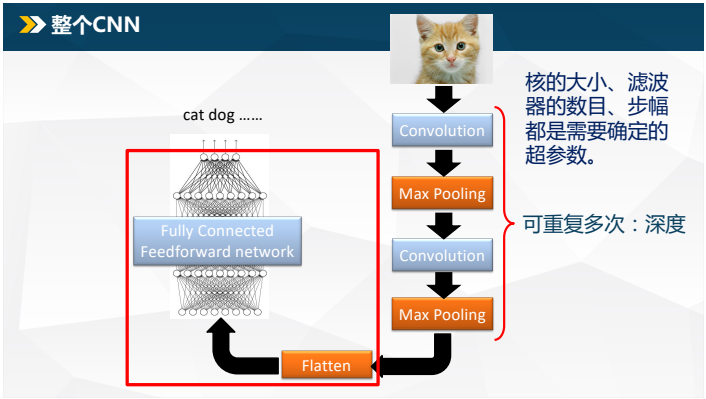
Example

designing a compact educational image classifier

Main idea

The durable lesson is staged spatial reduction with growing feature depth, not copying historical input sizes or classifiers.

Convolutional layers build a hierarchy of features



LeNet, AlexNet, and VGG design lessons: model and variables

Model and assumptions

LeNet established convolution-pooling hierarchies; AlexNet demonstrated scalable deep training; VGG used repeated small kernels for regular structure.

$$\mathrm{RF}(3 \times 3 \text{ then } 3 \times 3) = 5 \times 5, \qquad 2 \cdot 9C^2 < 25C^2$$

Symbol	Meaning	Status
stage	fixed spatial scale	several feature transforms
channel depth	feature capacity	typically grows after downsampling
small kernels	factorized context	more nonlinearities

LeNet, AlexNet, and VGG design lessons: worked example

Quantity	Value	Meaning
one 5x5 conv	$25C^2$	parameters
two 3x3 convs	$18C^2$	same nominal field
nonlinearities	2	richer mapping

What the result means

Reuse the hierarchical principle and adapt the head to the current task and resolution.

LeNet, AlexNet, and VGG design lessons: checking the result

Main idea

The durable lesson is staged spatial reduction with growing feature depth, not copying historical input sizes or classifiers.

What to check

validation accuracy per parameter and per FLOP

Limit

copying a classic dense classification head can dominate parameters and lock the model to one input size

LeNet, AlexNet, and VGG design lessons: implementation

1. divide the network into resolution stages
2. repeat small-kernel blocks
3. increase channels after downsampling
4. separate backbone and head
5. profile accuracy per compute

Check the result

Check validation accuracy per parameter and per FLOP on training and held-out examples.

Common mistake

copying a classic dense classification head can dominate parameters and lock the model to one input size

What breaks first in LeNet, AlexNet, and VGG design lessons

Example

designing a compact educational image classifier

Explain the calculation and check validation accuracy per parameter and per FLOP, using a small example.

Common mistake

copying a classic dense classification head can dominate parameters and lock the model to one input size

What to do

Reuse the hierarchical principle and adapt the head to the current task and resolution.

LeNet, AlexNet, and VGG design lessons: check your understanding

Question

Which ideas from early CNN families remain useful architectural principles?

Answer

The durable lesson is staged spatial reduction with growing feature depth, not copying historical input sizes or classifiers.

Decision. Reuse the hierarchical principle and adapt the head to the current task and resolution.

How can one stage represent patterns at multiple spatial scales without uncontrolled cost?



Parallel branches expose scale choices; one-by-one convolutions reduce channel cost before expensive spatial kernels.

Question

How can one stage represent patterns at multiple spatial scales without uncontrolled cost?

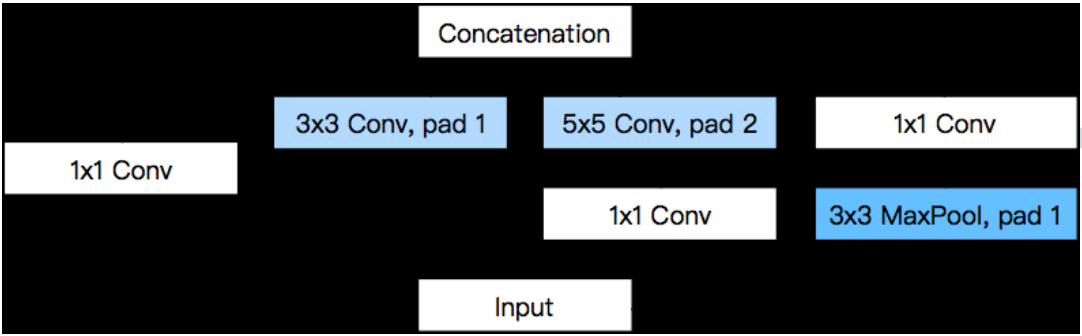
Example

classifying scenes containing both fine texture and large shapes

Main idea

Parallel branches expose scale choices; one-by-one convolutions reduce channel cost before expensive spatial kernels.

Inception combines several receptive-field sizes



Parallel branches use different receptive fields and concatenate channels.

Inception and multi-scale computation: model and variables

Model and assumptions

An Inception-style block applies parallel operations with different receptive fields and concatenates their channel outputs.

$$Y = \text{concat}\left(f_{1\times1}(X), f_{3\times3}(X), f_{5\times5}(X), f_{pool}(X)\right)$$

Symbol	Meaning	Status
$f_{k\times k}$	scale-specific branch	local context
1×1	channel projection	bottleneck or mixing
concat	channel concatenation	preserves branch outputs

Inception and multi-scale computation: worked example

Quantity	Value	Meaning
input channels	256	shared
1x1 reduction	256 $\rightarrow$ 64	before 5x5
spatial cost	4$\times$ lower	relative channel product

What the result means

Retain only branches that add validated scale-specific value under the target hardware profile.

Accept or reject Inception and multi-scale computation

Main idea

Parallel branches expose scale choices; one-by-one convolutions reduce channel cost before expensive spatial kernels.

What to check

scale-stratified accuracy and branch cost

Limit

parallel branches can add latency and memory traffic even when nominal FLOPs appear acceptable

Inception and multi-scale computation: implementation

1. identify relevant object scales
2. set branch output budgets
3. reduce channels before wide kernels
4. align spatial shapes
5. ablate each branch

Check the result

Check scale-stratified accuracy and branch cost on training and held-out examples.

Common mistake

parallel branches can add latency and memory traffic even when nominal FLOPs appear acceptable

The hidden assumption in Inception and multi-scale computation

Example

classifying scenes containing both fine texture and large shapes

Explain the calculation and check scale-stratified accuracy and branch cost, using a small example.

Common mistake

parallel branches can add latency and memory traffic even when nominal FLOPs appear acceptable

What to do

Retain only branches that add validated scale-specific value under the target hardware profile.

Inception and multi-scale computation: check your understanding

Question

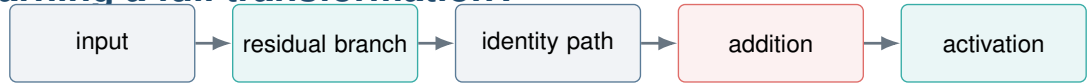
How can one stage represent patterns at multiple spatial scales without uncontrolled cost?

Answer

Parallel branches expose scale choices; one-by-one convolutions reduce channel cost before expensive spatial kernels.

Decision. Retain only branches that add validated scale-specific value under the target hardware profile.

Why does learning a residual often optimize more reliably than learning a full transformation?



The shortcut creates a direct information and gradient path, while the residual branch learns only the change needed from the current representation.

Question

Why does learning a residual often optimize more reliably than learning a full transformation?

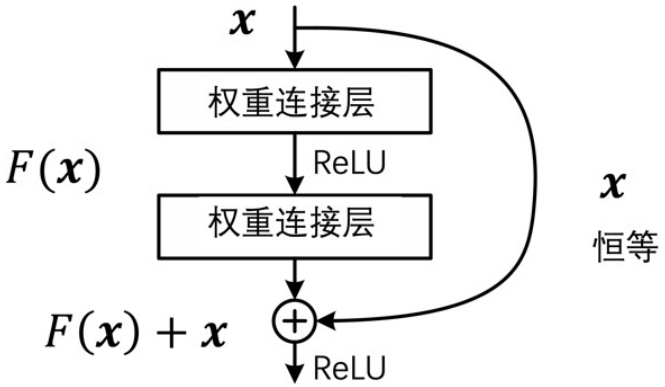
Example

deepening a VGG-like backbone without degrading optimization

Main idea

The shortcut creates a direct information and gradient path, while the residual branch learns only the change needed from the current representation.

A residual block adds an identity path



A residual block learns $F(x)$ and returns $F(x) + x$ before any following activation.

Residual learning and identity paths: model and variables

Model and assumptions

A residual block adds an identity or projected shortcut to a learned residual mapping.

$$y = F(x;\theta) + P(x), \qquad \frac{\partial y}{\partial x} = \frac{\partial F}{\partial x} + \frac{\partial P}{\partial x}$$

Symbol	Meaning	Status
F	residual branch	learned correction
P	identity or projection	shape-matching shortcut
y	block output	summed representation

Residual learning and identity paths: worked example

Quantity	Value	Meaning
same shape	$P(x) = x$	free shortcut
stride or channels change	$P =$ 1x1 conv	projection
initial $F \approx 0$	$y \approx x$	easy reference

What the result means

Use identity shortcuts when shapes agree and explicit projections when resolution or channel count changes.

Separate a good fit from a good decision in Residual learning and identity paths

Main idea

The shortcut creates a direct information and gradient path, while the residual branch learns only the change needed from the current representation.

What to check

gradient norms and validation gain with added depth

Limit

adding tensors with mismatched spatial alignment or channel semantics can silently corrupt the shortcut's meaning

Residual learning and identity paths: implementation

```
x = rng.normal(size=(2, 4, 4, 3))
residual = 0.05 * rng.normal(size=x.shape)
y_res = x + residual
print('Residual block shape/change:', y_res.shape, round(float(np.linalg.norm(y_res-x)), 4))
```


Evidence and failure check

Check gradient norms and validation gain with added depth with the fitted model and data split; reject the run if adding tensors with mismatched spatial alignment or channel semantics can silently corrupt the shortcut's meaning.

Do not trust Residual learning and identity paths before this check

Example

deepening a VGG-like backbone without degrading optimization

Explain the calculation and check gradient norms and validation gain with added depth, using a small example.

Common mistake

adding tensors with mismatched spatial alignment or channel semantics can silently corrupt the shortcut's meaning

What to do

Use identity shortcuts when shapes agree and explicit projections when resolution or channel count changes.

Residual learning and identity paths: check your understanding

Question

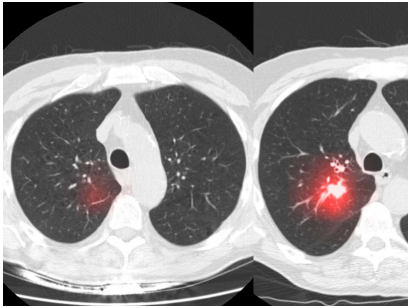
Why does learning a residual often optimize more reliably than learning a full transformation?

Answer

The shortcut creates a direct information and gradient path, while the residual branch learns only the change needed from the current representation.

Decision. Use identity shortcuts when shapes agree and explicit projections when resolution or channel count changes.

How should a shared visual backbone connect to classification, detection, segmentation, or clinical decisions?



Question

How should a shared visual backbone connect to classification, detection, segmentation, or clinical decisions?

Example

prioritizing suspected lesions in lung CT scans

Main idea

Global labels need pooled evidence; detection needs boxes; segmentation needs aligned dense outputs; clinical use also needs calibration and subgroup audits.

The task determines the output head

ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

- Classification: one class distribution for an image.
- Detection: class scores and bounding boxes for objects.
- Segmentation: a class distribution at each pixel.
- Compare errors at the level required by the task, including small objects and boundaries.



Task heads, error analysis, and vision deployment: model and variables

Model and assumptions

A task head converts spatial features into outputs at the label's grain, while evaluation preserves the deployment population and decision process.

$$\widehat{y} = h_{task}(F_{backbone}(X)), \qquad R = \mathbb{E}_{(X,Y)\sim P_{deploy}} L(Y, \widehat{y})$$

Symbol	Meaning	Status
$F_{backbone}$	visual representation	shared feature extractor
h_{task}	task head	matches label structure
P_{deploy}	deployment distribution	evaluation target

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Task heads, error analysis, and vision deployment: worked example

Quantity	Value	Meaning
classification	one label/image	global pooling
segmentation	one label/pixel	aligned decoder
detection	boxes + classes	local proposals or dense head

What the result means

Split at the independent acquisition unit and validate the complete decision pipeline.

Task heads, error analysis, and vision deployment: checking the result

Main idea

Global labels need pooled evidence; detection needs boxes; segmentation needs aligned dense outputs; clinical use also needs calibration and subgroup audits.

What to check

task metric, calibration, subgroup performance, and latency

Limit

randomly splitting frames or slices can place near-duplicates from one patient or drive in both training and test data

Task heads, error analysis, and vision deployment: implementation

1. match output grain to annotation
2. split by patient, site, or sequence
3. audit transformations visually
4. report subgroup and size metrics
5. monitor drift and calibration

Check the result

Check task metric, calibration, subgroup performance, and latency on training and held-out examples.

Common mistake

randomly splitting frames or slices can place near-duplicates from one patient or drive in both training and test data

Task heads, error analysis, and vision deployment: limitations

Example

prioritizing suspected lesions in lung CT scans

Explain the calculation and check task metric, calibration, subgroup performance, and latency, using a small example.

Common mistake

randomly splitting frames or slices can place near-duplicates from one patient or drive in both training and test data

What to do

Split at the independent acquisition unit and validate the complete decision pipeline.

Task heads, error analysis, and vision deployment: check your understanding

Question

How should a shared visual backbone connect to classification, detection, segmentation, or clinical decisions?

Answer

Global labels need pooled evidence; detection needs boxes; segmentation needs aligned dense outputs; clinical use also needs calibration and subgroup audits.

Decision. Split at the independent acquisition unit and validate the complete decision pipeline.

Compute the feature-map size before coding

Decision

Apply the padding, dilation, kernel, and stride formula independently to height and width.

Evidence

Verify the result with one tiny tensor whose output can be counted by hand.

Stop condition

Stop if the implementation silently rounds a non-integer output position.

Receptive field grows through the whole stack

Decision

Track receptive-field size and jump size layer by layer rather than reading only the last kernel size.

Evidence

Relate the final field of view to the physical scale of the target object.

Stop condition

Reject an architecture whose deepest unit still cannot see enough context for the label.

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

An augmentation is a claim about label invariance

Decision

For each crop, flip, color change, or geometric transform, state why the target should remain unchanged.

Evidence

Measure class-wise effects and inspect transformed samples with labels.

Stop condition

Remove an augmentation when it changes the clinical, physical, or semantic meaning of the example.

Saliency is a probe, not a proof

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Decision

Use saliency or class-activation maps to locate model sensitivity, then test the highlighted region with occlusion or counterfactual edits.

Evidence

Compare explanations across seeds and nearby checkpoints.

Stop condition

Do not interpret a smooth heat map as a causal explanation without intervention.

Deployment shift can be visible before accuracy drops

Decision

Monitor input quality, channel statistics, embedding drift, confidence, and delayed labels together.

Evidence

Keep slices for lighting, device, site, and rare classes.

Stop condition

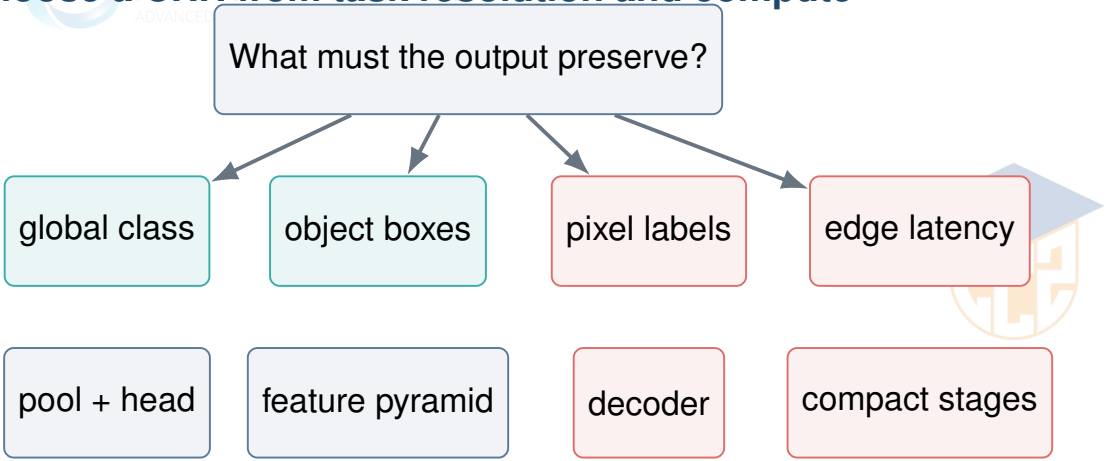
Escalate when drift affects a high-risk slice even if the overall average still looks stable.

Compare architectural operations by what they change

Operation	Preserves	Changes
stride-1 convolution	spatial indexing	local features and channels
pooling / stride	coarse arrangement	resolution and context
Inception branch	shared input grid	parallel scale features
residual block	shortcut representation	learned correction



Choose a CNN from task resolution and compute

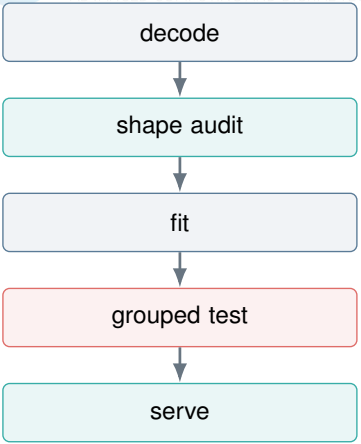


Start from the deliverable

Start with the smallest relevant object and label grain, then allocate spatial stages, channel depth, shortcuts, and the task head.

A reliable vision workflow

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER



1. version decoding and preprocessing
2. calculate every output shape and receptive field
3. visualize transformed samples and feature maps
4. split by acquisition unit
5. report scale and subgroup metrics
6. profile the complete served pipeline

A valid tensor can still be a semantically invalid image.

Connect architectural changes to observable failures

$$\text{utility} = \text{task score} - \lambda_t \,\text{latency} - \lambda_m \,\text{memory}$$

Observation	Likely issue	Next action
small objects missed	early downsampling	retain higher resolution
train good; new site poor	domain shift	audit preprocessing and data
skip addition fails	shape alignment	calculate projection

Stress-test visual behavior

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

spatial fidelity | domain robustness | system budget

- translate, crop, blur, and rescale inputs
- stratify by object size and location
- inspect post-transform images
- compare sites, sensors, or patients
- measure calibration and abstention
- profile memory and tail latency

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Six takeaways to retain

1. images are spatial tensors
2. convolution shares local detectors
3. shape arithmetic defines alignment
4. channels encode a feature bank
5. downsampling trades detail for context
6. task heads must match label grain

Exit ticket: repair a failing vision model

Prompt

Propose one architectural change and one validation test. State the affected spatial scale, expected tensor shape, task metric, and deployment cost.

Case

A classifier performs well overall but misses small lesions near image boundaries. The backbone downsamples by eight before its second residual stage.